

Floating point Numbers.

Normalized Form

$$A = (-1)^s \times P \times 2^x \quad \left(P = 1 + M, 0 \leq M < 1, x \in \mathbb{Z} \right)$$

M $\rightarrow$ Mantissa

x $\rightarrow$ exponent

z $\rightarrow$ set of integers.

significand
betⁿ 1 & 2

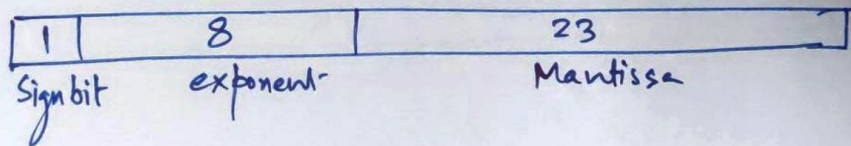
IEEE 754 format

1'xxxxx
M

1
Sign

8
exponent

23
Mantissa



bias $\rightarrow 127$

$$E = X + 127$$

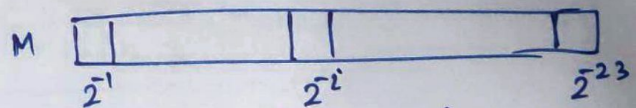
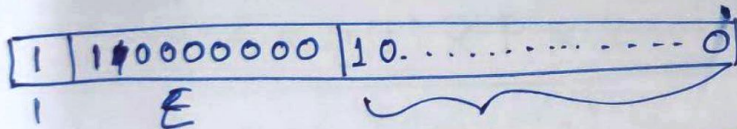
(0-255) $\uparrow$ -127 to +128

Normalized numbers.

$$X \rightarrow -126 \text{ to } +127$$

$$A = (-1)^S \times P \times 2^{E - \text{bias}}$$

$$-3 = (-1)^1 \times (1 + .5) \Rightarrow \boxed{(-1)^1 \times (1 + 2^{-1}) \times 2^1}$$



$$M = \sum_{i=1}^{23} x_i \cdot 2^{-i}$$

1011001

0.0001201
 1.4101×2^{-4}

1.011001 $\times 2^6$
Significand

$$A = P \times 2^x, \quad \left(P = 1 + M, \quad 0 \leq M < 1, \quad x \in \mathbb{Z} \right)$$

↑
Mantissa

$A = (-1)^s \times P \times 2^x$

IEEE 754 format.

Sign(s)	Exponent(x)	Mantissa (M)
1	8	23

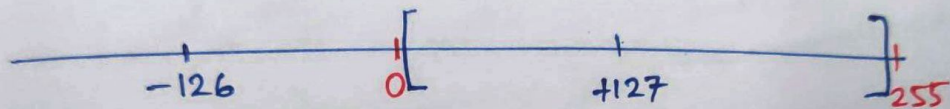
$E \rightarrow 0$ & 255 are reserved.

$E \rightarrow 1$ & 254

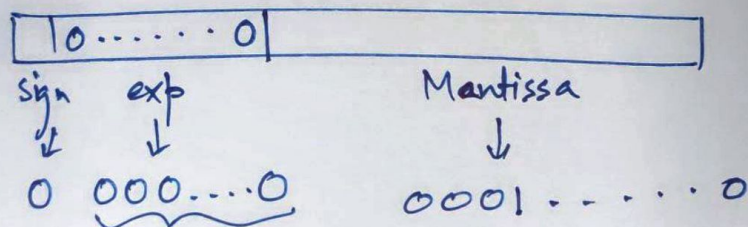
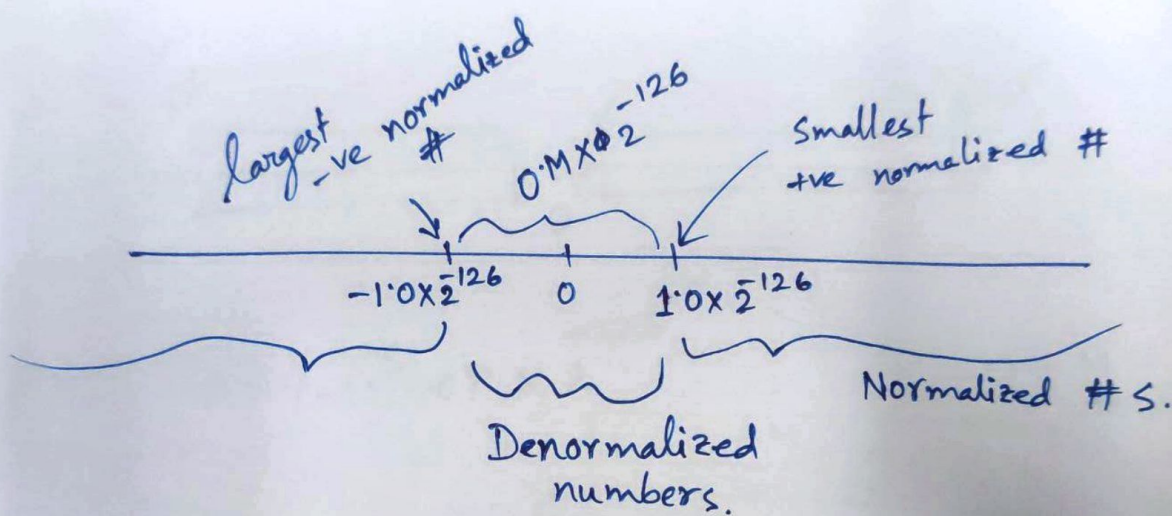
$E \neq X + \text{bias}$.

$$A = (-1)^S \times P \times 2^{E - \text{bias}^{127}}, \quad (P = 1 + M, 0 \leq M < 1, 1 \leq E \leq 254)$$

<u>E</u>	<u>M</u>	<u>Value</u>
255	0	∞ if $s=0$
255	0	$-\infty$ if $s=1$
255	$\neq 0$	NAN (not a number)
0	0	0
0	$\neq 0$	Denormal number.



For normalized values, you will never find exponent field of all 0's or all 1's.

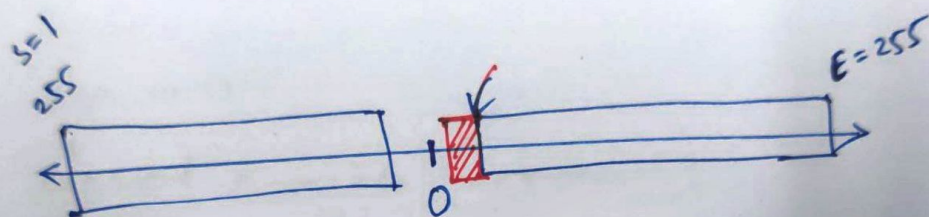


$$\text{Mantissa} = 0001 \dots$$

$$= 0 + 2^{-4}$$

$$\text{Answer} = 2^{-4} \times 2^{-126}$$

$$= 2^{-130}$$



$$A = (-1)^S \times P \times 2^{-126}$$

$$P = 0 + \underline{M}$$

$$P = \underline{0 + M} \quad 0 \leq M < 1$$

$$2^{-126} \div 2$$

$$2^{-146}$$

sign $\rightarrow 0$

exponent $\rightarrow -126$ (denormal)

0 0000 0000 0000 0000 0000 0000 0000 $\dots 2^{-20} \dots$

Double Precision

S $\rightarrow 1$

E $\rightarrow 11$

M $\rightarrow 52$

Assembly Language } family of low level programming languages.

- Low level programming language
- Specific for an ISA

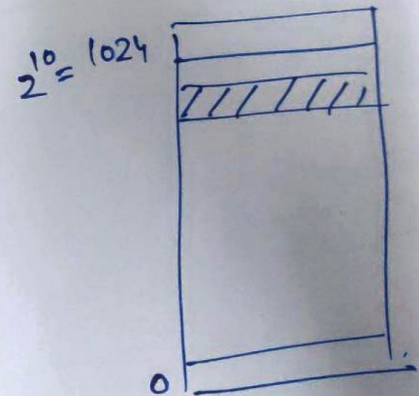
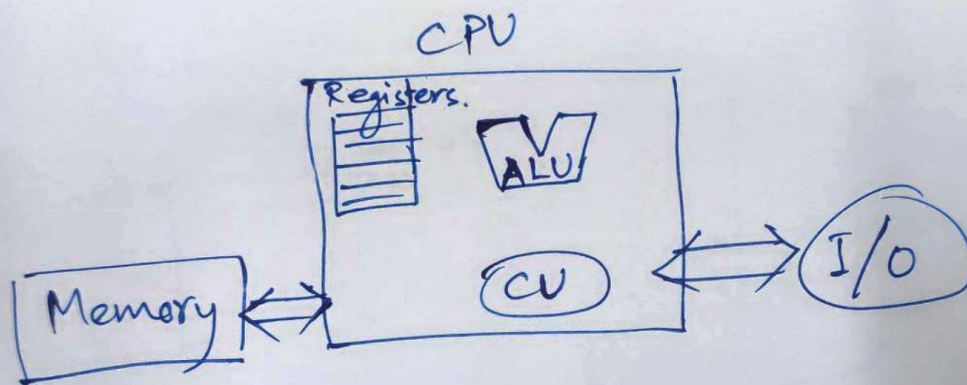
Why learn Assembly language?

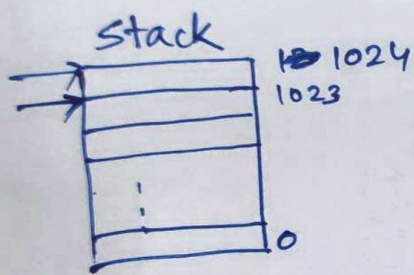
- Highly efficient code
- IoT platforms.

Two parts:

1. Instruction code/opcode
2. a list of operands.

* Where do operands come from & where do results go?





$C = A + B.$

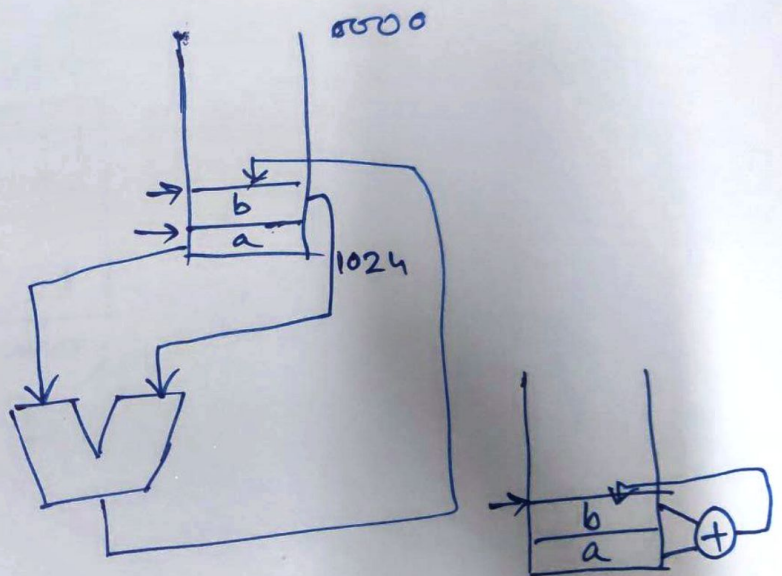
Push A

Push B.

Add.

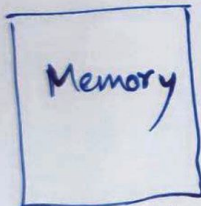
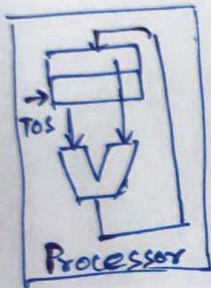
~~Pop C.~~

Pop C.



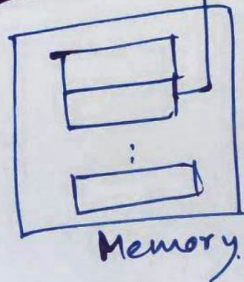
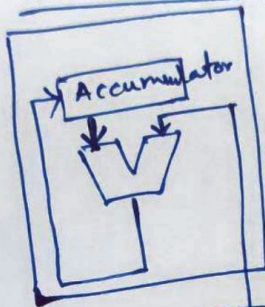
Machine Models.

Stack



of explicitly
named operand '0'.

Accumulator



'1'